

Report of Fakeddit Data Analysis

by Ivan Polishchuk & Paweł Moćko

Introduction & Motivation

The problem of fake news dissemination is acute in modern society, directly threatening citizens' trust in institutions and provoking irrational panic during critical moments such as medical epidemics. Combating this threat requires reliable machine learning algorithms, yet their development is seriously hindered by a shortage of high-quality data. Most traditional datasets, such as LIAR or Some-Like-It-Hoax, have very substantial limitations that make them largely unsuitable for comprehensive analysis. As a rule, they are too small in volume, restricted exclusively to text format and binary classification logic (where a news item can only be true or false), and focused on very narrow subject areas, which prevents models from learning from diverse everyday content.

Dataset Characteristics

The uniqueness of the Fakeddit dataset lies in its multimodality, which allows algorithms to rely not only on isolated text but also on the visual context of images, significantly enriching the amount of information that can be used for the detection of misinformation. The data consists of post titles, images associated with them and additional metadata such as creation time, number of comments, subreddit name, etc. The authors of the original dataset carefully extracted not only the headlines and attached images but also rich metadata, including scores, author names, and audience comments.

The dataset consists of over 1 million posts, of which around 682k have an image associated with them. The posts come from across cleanest and most relevant 22 subreddits. The overall quality of the final general sample was good enough, no further pre-processing was required over the one provided by the dataset authors. During data collection there were also additional quality assurance steps, which can be in practice reduced to all posts with score < 1 and subreddit-specific title tags removing.

Labels of The Posts

Labeling the vast amount of data was performed using the distant supervision method (an approach that involves automatically assigning labels to all posts based on the known general theme of the subreddit from which they were extracted). This eliminates the need to manually label each individual post. Thus, for each sample there are three different labels assigned, corresponding to different levels of granularity. We will describe it here vividly, but concisely:

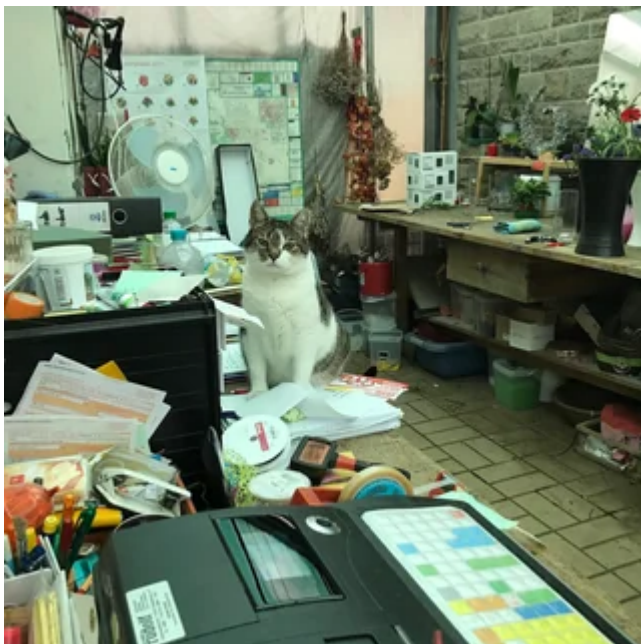
- 2-way labels: a **True/False** split - represents whether or not the posts presents factual information
- 3-way labels: posts containing false data have been split into two categories:
 - false text, false sample - sample has **False** label in 2-way labeling and contains untrue information
 - true text, false sample - sample has been deemed **False**, but contains "true" text - it involves only posts containing propaganda posters.
- 6-way labels: **True** label remains the same as in the other two kinds of labeling, the **False** label has been split into:
 - imposter content; i.e. content auto-generated by bots

- misleading content: content conveyed in a manipulated way to mislead people in believing false narratives
- satire / parody
- false connection: submission images in this category do not accurately support their text descriptions
- manipulated content: content that has been manually altered e.g. via image modifications

It should be noted that some of these categories, in particular Manipulated Content, consist almost entirely of visually altered pictures, so when creating narrowly focused models that analyse only textual data, it is advisable to disregard such labels so as not to introduce unnecessary mathematical noise into the training sample. Publicly available error analysis conducted by original creators of dataset revealed that models struggle the most with bot-generated content and satire (the issue of real-world knowledge), while manipulated images are surprisingly easy to catch though a bias toward the "True" label remains due to class imbalance.

Also, for each subreddit authors of data have inspected 10 random samples. If the 6-way label they assigned for each of the sample matched, so then the subreddit was kept in the dataset and the label was assigned to each post in the subreddit. This means the misinformation classification task turns into just predicting the subreddit based on the title alone. On the other hand, a curious example is connected with the label "manipulated" for the subreddit `psbattle_artwork`. That subreddit does contain image manipulation, but the posts there are mostly satirical in nature.

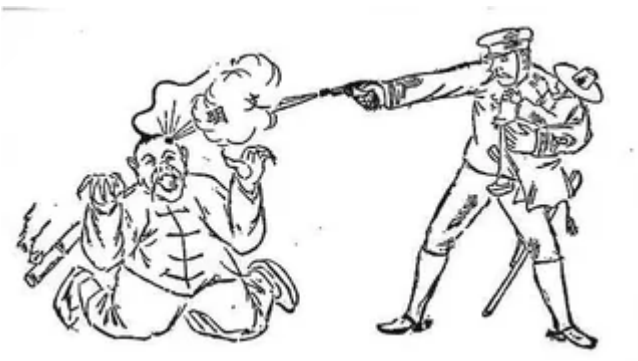
Some Indicative Examples



Above is a picture of a cat titled "He looks huge". This post comes from `confusing_perspective` subreddit - thus it's labeled as false connection.



The above post is titled "Obligatory godzilla post" and comes from [psbattle_artwork](#) subreddit.



Cleaned title: "Newspaper illustration depicting japan shooting china with the bullet of civilization first sinojapanese war"



Another propaganda poster "Roosevelt figured wrong the tentacles of the dollaroctopus will be cut the jews in the white house and the gold in fort knox have been surrounded by young people by the armies of labor nsb poster from the netherlands"

Proposed Pipeline & Workflow

To formalise our project, we need to describe the structure and order of implemented data manipulations. The creators of the original Fakeddit were extremely concise in their assessments in article "r/Fakeddit: A New Multimodal Benchmark Dataset for Fine-grained Fake News Detection". The final metrics are presented rather sparingly, which complicates direct analysis. Therefore, this project offers an alternative approach. We would like to present here not merely a technical replication but an attempt to deepen and conceptually rethink the conclusions of original creators. The main business task is, hence, to independently verify the results obtained by original authors. We strive to build alternative methods for constructing classifiers and to see whether these new, more modern approaches can provide serious competition in real world conditions. The implementation of our plan differs qualitatively from that proposed by the original due to strict objective constraints. The main challenge was the colossal size of the image database, exceeding 100gb. Operating with such multimedia arrays on basic computing resources is very difficult. Moreover, we deliberately decided to raise the complexity bar even more, abandoning the initially considered simplified version of the dataset containing only 55k items, concentrating on a huge sample from dataset crafted on Kaggle (700k records). An important consequence of this decision became a key feature of our pipeline. **The vast majority of analytical work is carried out exclusively on textual data. Adding visual content, thus, became the final stage of our project.**

We built a strict sequence of dataset analysis consisting of five evolving stages. The first part involves exploratory **data analysis (EDA) and data cleaning**. Any metrics without this kind of step would be distorted by information noise, so obtaining a clean base of 700k items became the foundation for all further actions. In

the next stage we create **baseline models based on TF-IDF**. The baseline relies on primitive word counting and completely ignores the semantics of language, but business always needs a starting point, the simplest and fastest algorithm against which all expensive and complex solutions will be compared. Understanding this conceptual limitation naturally leads to the third stage: testing **advanced ensemble models in combination with modern language transformers**. Here we use embedders to extract deep meaning from texts and feed these complex data to algorithms such as LightGBM. The issue of tuning is being considered next. So, using the **Optuna framework** we basically tune hyperparameters to the specific geometry of our analysed information. Finally, as we are aware, that text alone models cannot evaluate properly the situation with tricky text-picture interplay, the **Vision-Language Model was considered and used on the data**.

Each stage of our pipeline is driven by a set of general assumptions (formally, hypotheses). First, we expect to achieve results close to those obtained in the original study, assuming that our overwhelming focus on textual data in the first four stages will not lead to a radical drop in metrics compared to the overall benchmark. Finally, the original paper leaves some ambiguity regarding how critical the presence of images is for the claimed fine-grained classification. We assume some particular importance of visual content. Therefore, our final business hypothesis is that **the image-based classification model will show a result at least comparable to the best purely text-based model**, and ideally surpass it.

Teoretical Fundamentals

The processing of Fakeddit dataset could be understood as **supervised learning task**. We aim to build a model that uncovers hidden patterns and predicts one of six categories for new posts. From this perspective, it should be reminded, that classification technique basically finds separating boundaries in a described feature space. Since we have six classes, we use strategies as native multiclass algorithms. Before mere modelling, we perform Exploratory Data Analysis (EDA) using descriptive statistics and visualizations to detect class imbalance, text length distributions, and potential outliers. Then we data cleaning, which is critical to prevent data leakage problems, where test information accidentally enters training.

Our meaningful benchmark consists of TF-IDF vectorization combined with logistic regression. **TF-IDF** converts text into a sparse numerical matrix by counting term frequencies and penalizing words that appear too often across the corpus, thereby highlighting truly distinctive terms. **Logistic regression** then takes these vectors, computes a weighted sum of features, and passes it through a sigmoid function to produce a probability of a post being fake. This baseline is mathematically transparent, fast and allows us to directly inspect which words the model considers deceptive. But, linear models often fail to capture non-linear patterns in language, so we turn to ensemble methods, specifically Random Forest and LightGBM. **Random Forest** builds many independent decision trees in parallel, reducing variance and overfitting. **LightGBM**, a gradient boosting algorithm, sequentially adds trees that correct the errors of previous ones by approximating the antigradient of the loss function. Its histogram-based discretization and leaf-wise tree growth make it exceptionally fast and accurate on large datasets.

It should be said that fundamental limitation of TF-IDF is that it ignores word order and context, treating each text as a mere bag of words. To capture semantic meaning, we use **language embeddings from SentenceTransformers**. These dense vectors encode entire sentences such that geometrically close vectors indicate semantically similar texts, even without shared vocabulary. We employ lightweight models like all-MiniLM-L12-v2 and paraphrase-MiniLM-L3-v2, which are products of knowledge distillation. Instead of relying on default hyperparameters, we use **Optuna, a Bayesian optimization framework based on the TPE** (Tree-structured Parzen Estimator). Optuna does not blindly search but mathematically analyses previous trials to select the most promising hyperparameter values. As an example, for our baseline, we tune the TF-IDF

parameters (max_df, min_df) and the logistic regression regularization strength. For LightGBM, we optimise the number of trees, learning rate, tree depth, and leaf count.

As it was said before, in our project we try modern **Vision-Language Models (VLMs)**, which go far beyond traditional convolutional networks by deeply integrating visual and textual perception. At their core are so-called Vision Transformers that split an image into small patches, convert them into numbers, and project them into the same hidden space as text tokens. Cross-attention layers allow the model to compare written words with specific objects in the image. The model processes both an image and the post's headline simultaneously, then extracting the last layer's hidden states, applying an attention mask to filter padding, and average the results to obtain a single fused vector that combines the meaning of text and image in the end. This fused representation can be theoretically crucial because of the detection enabling of the most difficult category, False Connection, where a truthful text and a ambiguous photo are deliberately paired to create a some type of compromised story.

Step I. Introductory EDA & Cleaning

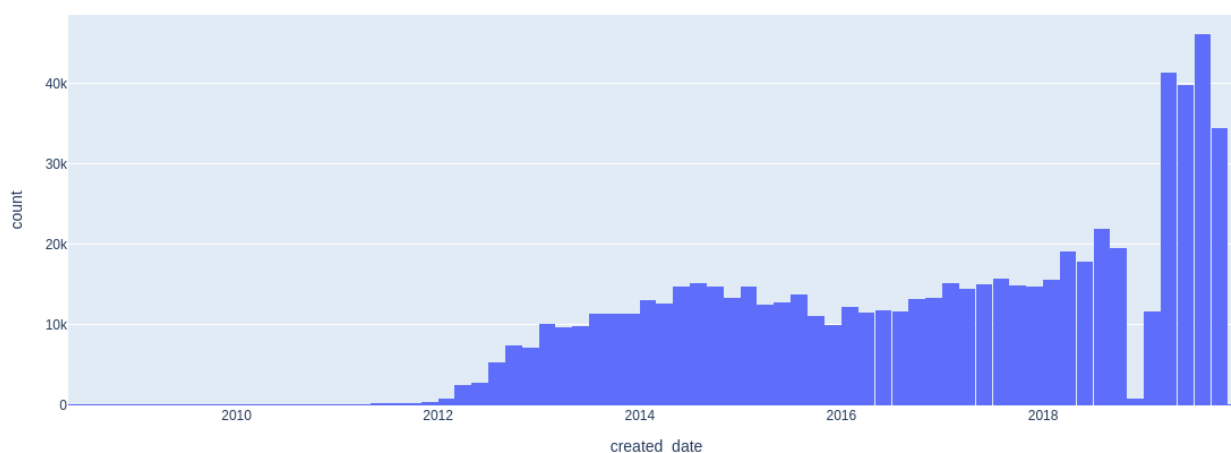
Data Loading & Environment

The practical implementation of our pipeline begins with loading and examining the raw dataset. Data acquisition was carried out along two parallel tracks: loading directly into the Kaggle cloud environment to ensure access to necessary computing resources, and local deployment of a subsample for rapid baseline testing. To handle a dataset of this complexity efficiently, we deliberately abandoned Pandas in favor of **Polars**. Written in Rust, this module supports process parallelization, which provided even better loading and transformation speeds.

Temporal Resolution

The dataset covers a large time span, with the first posts dating back to 2008 and the most recent from 2019. The chronological distribution shows a steady increase in platform activity over the years, reaching its peak towards the end of 2019.

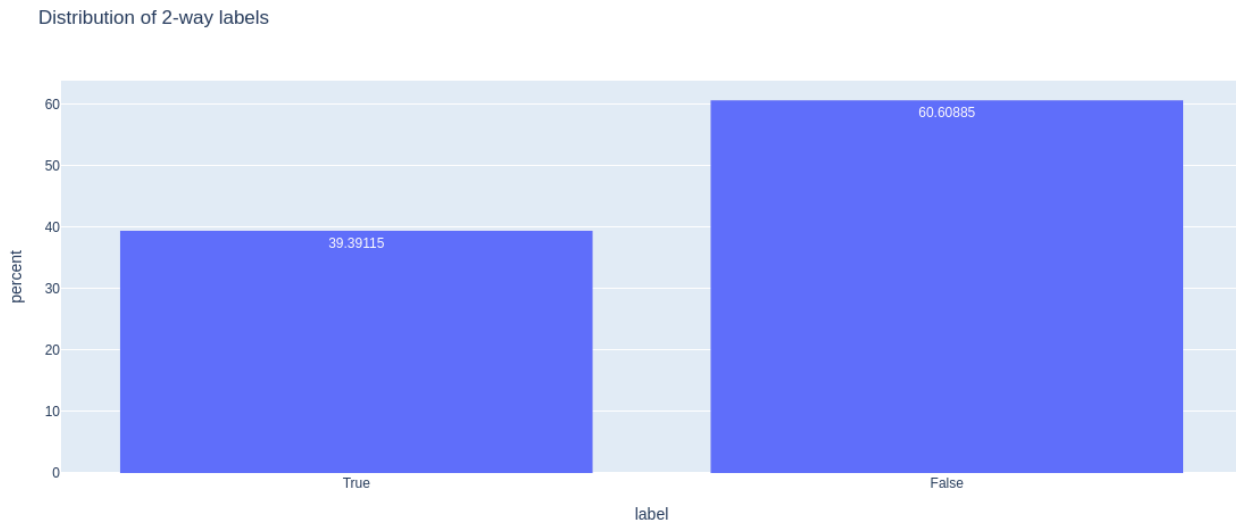
Post distribution over time



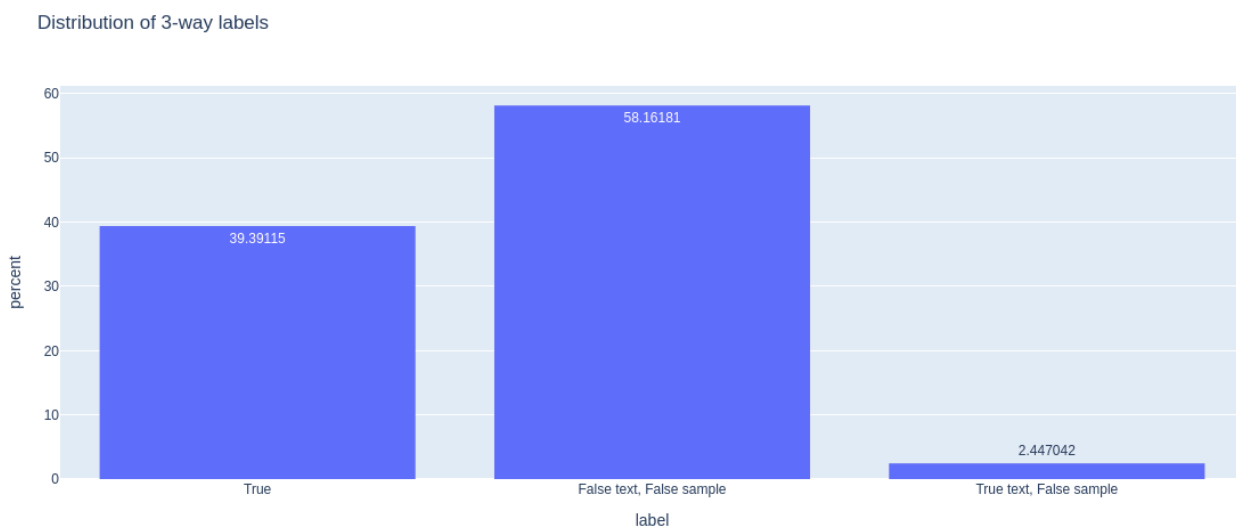
Distribution & Data Leakage

Through our structural exploration, we identified 17 original columns, encompassing identifiers, platform metadata, media links, textual content, and three target variables. The target variables shows a significant imbalance:

- 2-way classification: ~60% false posts vs. 40% truthful.

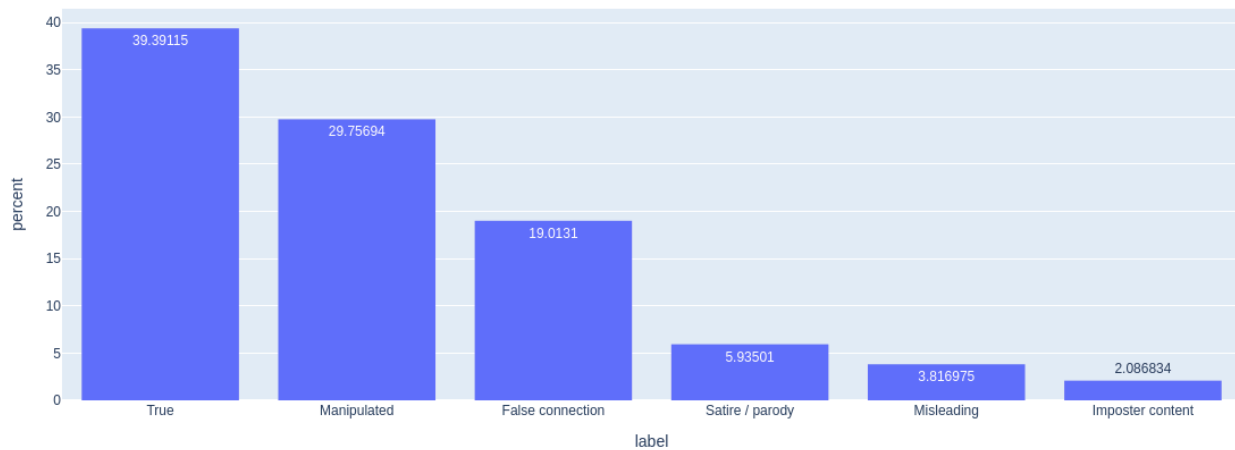


- 3-way classification: ~58% completely false, 40% truthful, and ~2% fake posts containing truthful text.



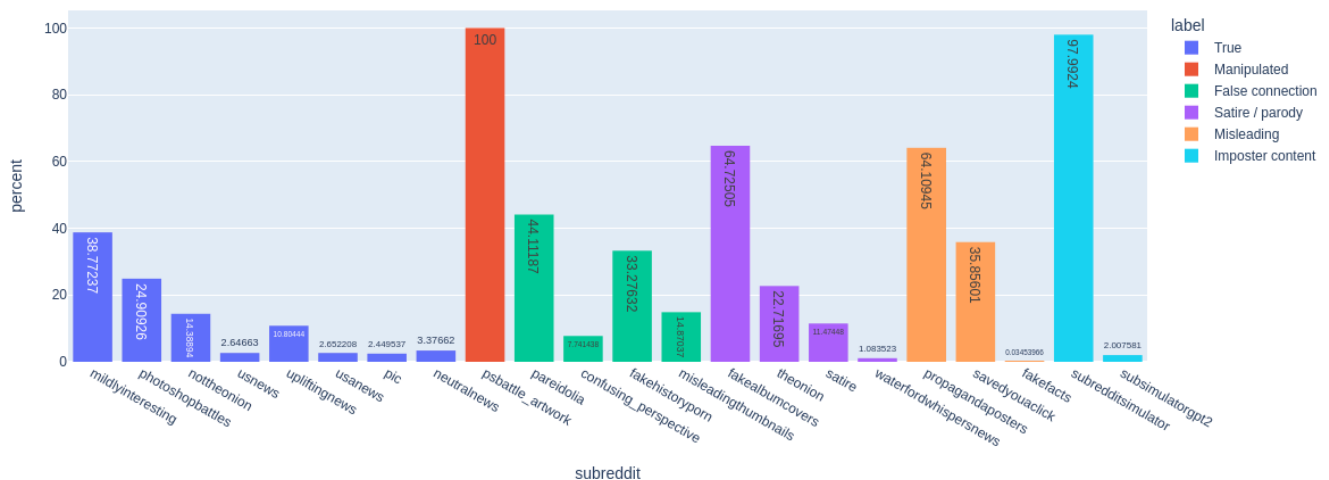
- 6-way classification: Truthful news (39%), visually manipulative content (30%), false connection (19%), satire (6%), misleading content (4%), and imposter content (~2%).

Distribution of 6-way labels



A critical discovery during the EDA stage was the rigid almost 100% dependency between the fake category and the subreddit community it was posted in. If we had retained the subreddit column as a training feature, the algorithm would simply memorize the association between specific subreddits and labels rather than learning linguistic or visual patterns. To prevent this destructive data leakage, the subreddit column, along with author names and domains, was irrevocably removed from the pipeline.

Distribution of 6-way labels

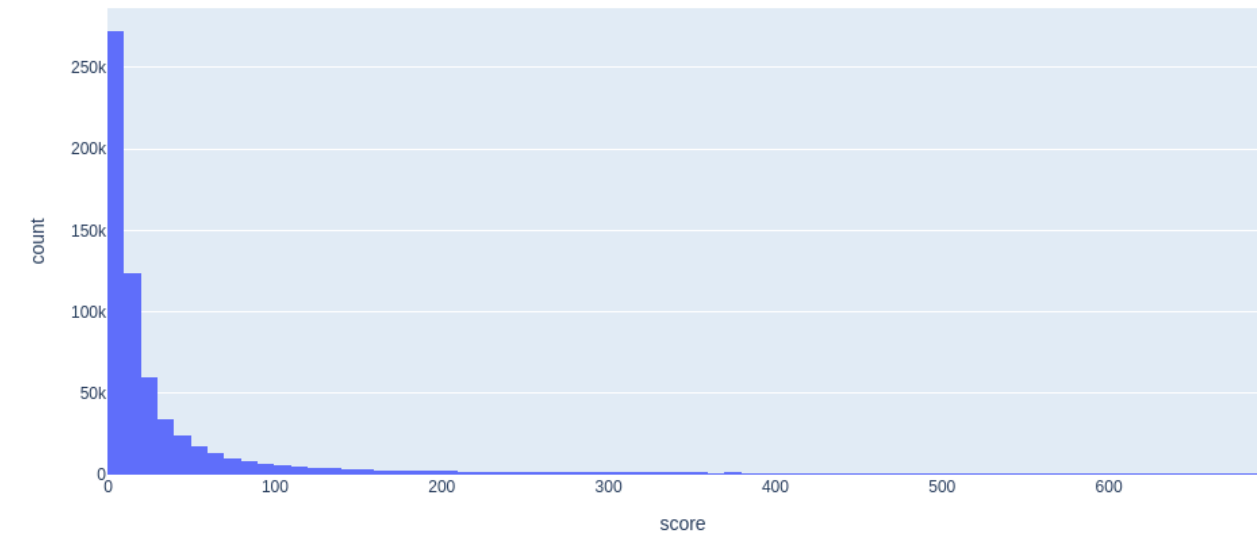


Provided & Synthesized Metadata

The dataset also provides rich platform metadata, including scores (upvotes minus downvotes) and comment counts. As expected, user scores demonstrate an exponential decline: the median score is only 14 points, with only an extremely small fraction of posts going viral.

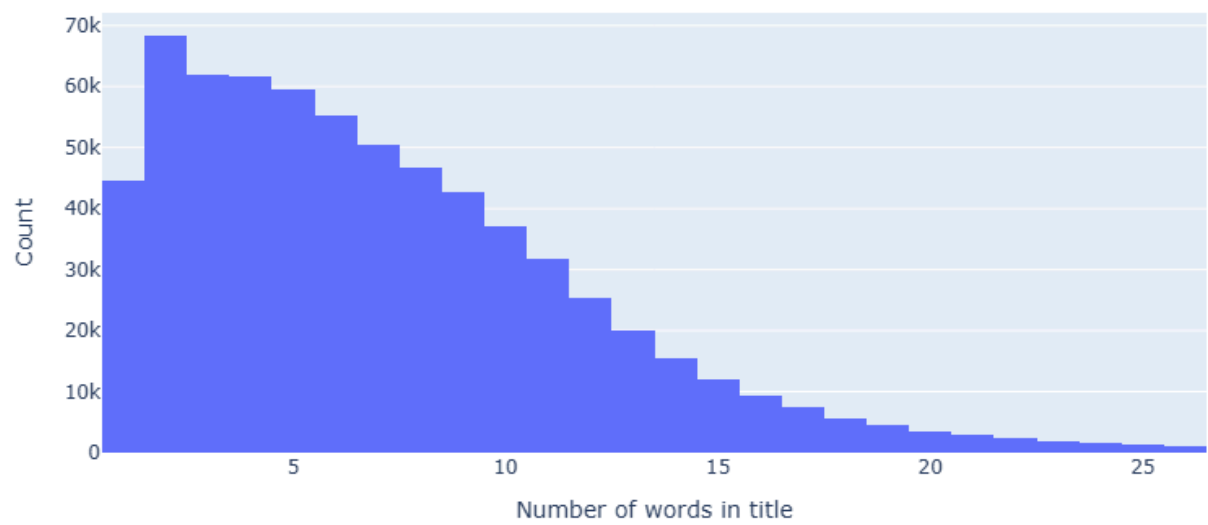
column	null_count	null_pct
str	u32	f64
"author"	34841	0.051037
"clean_title"	0	0.0
"created_utc"	0	0.0
"domain"	203139	0.297569
"hasImage"	0	0.0
"id"	0	0.0
"image_url"	1863	0.002729
"linked_submission_id"	479522	0.702431
"num_comments"	203139	0.297569
"score"	0	0.0
"subreddit"	0	0.0
"title"	0	0.0
"upvote_ratio"	203139	0.297569
"2_way_label"	0	0.0
"3_way_label"	0	0.0
"6_way_label"	0	0.0
"created_date"	0	0.0

Distribution of scores for subreddit posts



We also visualized the text length distribution. The vast majority of headlines contain between 2 and 10 words, after which the density drops sharply.

Distribution of cleaned title word counts



Using all or significant part of dataset's metadata could be tricky. For instance, there are almost 35,000 different authors, and most appear only once or twice, though a few clear outliers exist which could theoretically provide some influence on result. We also analyzed the matrix of missing values. Metadata, thus, was deemed secondary to our primary semantic and multimodal analysis goals, there was no complex imputation strategy needed for these nulls, because they were handled via our general cleaning steps.

statistic	author	post_count
str	str	f64
"count"	"274605"	274606.0
"null_count"	"1"	0.0
"mean"	null	2.485965
"std"	null	76.171882
"min"	"-----_---"	1.0
"25%"	null	1.0
"50%"	null	1.0
"75%"	null	2.0
"max"	"zzzzz94"	34841.0

Data Cleaning & Deduplication

We applied an exact duplicate search by grouping the array by the unique combination of the cleaned title (clean_title) and image URL (image_url), keeping only the first unique occurrence. This eliminated information noise and prevented models from overfitting on repeated posts. Moreover, analyzing the missing values revealed over 201,000 gaps in the comment and score columns. We found that the vast majority of these anomalies strongly correlated with the subreddit psbattle_artwork, which has been removed from Reddit. So,

we completely deleted all rows associated with this inactive messy part, ridding of a massive array of nulls and removing content whose primary veracity can no longer be independently verified.

Splitting The Data

For the purpose of training, we split the data using a 60/20/20 stratified proportion. While a temporal split initially seemed justified, the labeling strategy used by the dataset creators means that time is not the main source of bias. For example, posts in the `usnews_subreddit` are automatically assigned as truthful regardless of the date. Therefore, stratified splitting ensures that all classification models receive a proportionally representative sample of the complex 6-way labels.

Step II. The Power & Unility of Just-Text Models

Here we move to the practical implementation and evaluation for our text-based classifiers. For these experiments, we exclusively used the `clean_title` column. It should be noticed, that the baseline model metrics creates benchmark we will attempt to surpass using more complex architectures. Implemented TF-IDF vectorizer searches for uni- and bigrams with a limit of 30,000 features.

The Binary Case

Firstly, we evaluated the model's performance on 2-way labels (truth versus false) to gain a better intuition of how predictions are made. The algorithm really has the potential to confidently separate the classes. Next, to understand the internal logic of this binary classifier, we extracted the top 10 words with the highest predictive weights for each category. It should be also considered that the model is biased towards the most common label (False).

Confusion Matrix - Binary Model

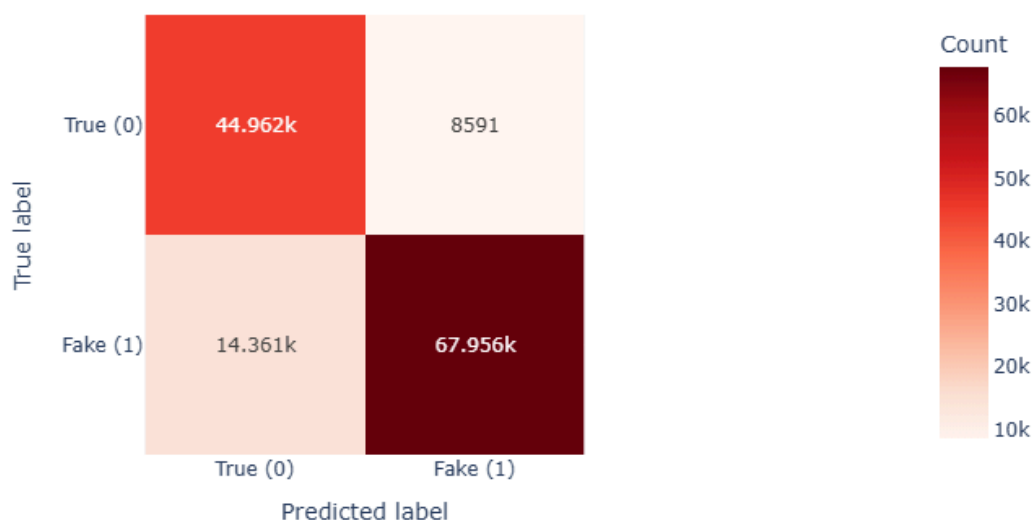


Table: Classification metrics

Class	Precision	Recall	F1-score	Support
True (0)	0.7579	0.8396	0.7967	53553

Class	Precision	Recall	F1-score	Support
Fake (1)	0.8878	0.8255	0.8555	82317
Accuracy			0.8311	135870
ROC AUC (Binary)	0.9085			

Table: Top-10 words/phrases for True and Fake classes

Rank	True	Fake
1	says	cutouts
2	in	circa
3	police	other discussions
4	donates	discussions
5	sign	mrw
6	lets you	colourized
7	saves	til
8	way the	florida man
9	tells	colorised
10	these	poster

We get an interesting overview by the ranking of words that are the most impactful for the predictions. Then, to get more context we took a look at the most common words for each subreddit:

```
[30...] shape: (22, 11)
```

subreddit	rank1	rank2	rank3	rank4	rank5	rank6	rank7	rank8	rank9	rank10
str	str	str	str	str	str	str	str	str	str	str
"confusing_perspective"	"the"	"this"	"like"	"looks"	"and"	"with"	"giant"	"look"	"dog"	"cat"
"fakealbumcovers"	"the"	"and"	"you"	"self"	"titled"	"this"	"for"	"album"	"selftitled"	"with"
"fakefacts"	"the"	"its"	"and"	"that"	"know"	"actually"	"size"	"fake"	"facts"	"had"
"fakehistoryporn"	"the"	"circa"	"his"	"and"	"for"	"after"	"first"	"from"	"before"	"with"
"mildlyinteresting"	"the"	"this"	"and"	"like"	"has"	"found"	"that"	"with"	"looks"	"for"
"misleadingthumbnails"	"the"	"this"	"with"	"new"	"from"	"and"	"for"	"out"	"man"	"dog"
"neutralnews"	"trump"	"the"	"for"	"and"	"says"	"with"	"after"	"from"	"new"	"over"
"nottheonion"	"for"	"the"	"man"	"with"	"after"	"and"	"police"	"from"	"says"	"woman"
"pareidolia"	"this"	"the"	"happy"	"like"	"looks"	"you"	"guy"	"little"	"was"	"and"
"photoshopbattles"	"this"	"the"	"with"	"cat"	"dog"	"and"	"his"	"man"	"guy"	"from"
"pic"	"the"	"and"	"this"	"from"	"sunset"	"park"	"lake"	"new"	"for"	"national"
"propaganda posters"	"the"	"poster"	"and"	"for"	"war"	"from"	"you"	"soviet"	"with"	"ussr"
"psbattle_artwork"	"the"	"you"	"this"	"other"	"discussions"	"cutouts"	"and"	"for"	"here"	"not"
"satire"	"the"	"for"	"trump"	"and"	"with"	"new"	"after"	"from"	"man"	"you"
"savedyouaclick"	"the"	"this"	"you"	"and"	"what"	"why"	"for"	"how"	"are"	"one"
"subredditsimulator"	"the"	"and"	"this"	"for"	"with"	"you"	"that"	"was"	"from"	"have"
"subsimulatorgpt2"	"the"	"and"	"this"	"with"	"for"	"you"	"was"	"that"	"not"	"skeltal"
"theonion"	"the"	"for"	"and"	"with"	"this"	"that"	"life"	"you"	"have"	"out"
"upliftingnews"	"the"	"for"	"and"	"with"	"from"	"after"	"his"	"man"	"who"	"her"
"usanews"	"the"	"for"	"and"	"trump"	"with"	"new"	"after"	"from"	"police"	"are"
"usnews"	"the"	"trump"	"for"	"and"	"with"	"after"	"from"	"new"	"says"	"man"
"waterfordwhispersnews"	"the"	"for"	"local"	"man"	"new"	"you"	"from"	"with"	"this"	"about"

For truthful posts, the algorithm highlights markers typical of a dry news style (says, police, donates). In contrast, fake content is dominated by specific internet slang (mrw, til) and technical tags (cutouts, colourized). It is characteristic that the model relies not so much on the deep semantics of deception, but on explicit artifacts of the Reddit environment. Terms like circa or colorised heavily imply the fakehistoryporn subreddit, while cutouts points to psbattle_artwork. This confirms that term frequency provides indirect information about the subreddit, compromising some form of data leakage.

6-way case

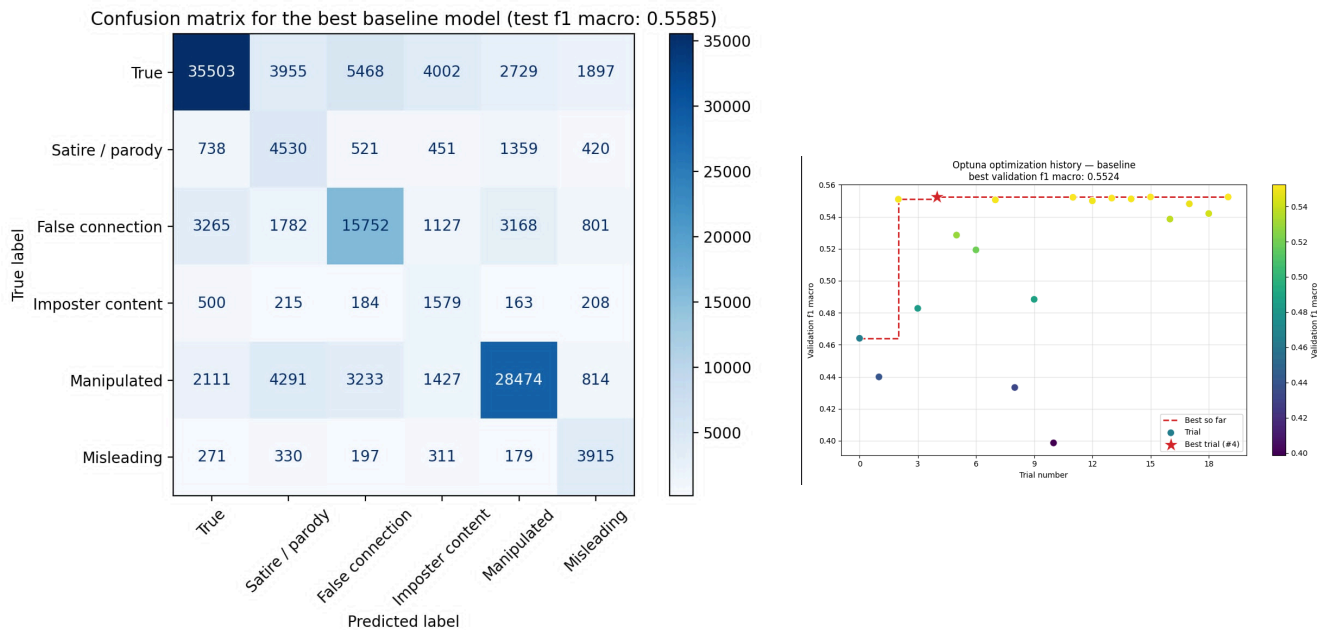
Then we applied exactly the same sequence of actions to build the 6-class model. As expected, when moving to a more robust classification, the absolute metric values noticeably decreased, though the probabilistic ROC-AUC metric remained at a stable, high level (0.9003).

The confusion matrix (presented below) clearly demonstrates the reason for the drop in accuracy. While the True and Manipulated classes are predicted quite confidently, strong confusion arises among the remaining categories. The algorithm makes massive errors when distinguishing satire, imposter content and misleading posts, often unfairly classifying them as truthful news. Logistic regression with TF-IDF works as a qualitative keyword filter, but it is incapable of understanding irony or contextual mismatches. The binary model is unconditionally superior in raw metrics, but it is too simplistic for real-world information analysis. The 6-class model represents a much more profound categorisation. Consequently, the performance of the 6-class baseline model, also hyperparameters-optimised in Optuna by 20 trials, establishes the main benchmark for the project.

Table. The Baseline Model (already tuned by Optuna)

Confusion Matrix

Optuna training history



The word ranking has been also provided in this case:

	rank	True	Satire / parody	False connection	Imposter content	Manipulated	Misleading
0	1	lets you	questions with	circa	mrw	cutouts	poster
1	2	uplifting	must see	colourized	til other discussions		posters
2	3	says	blog	colorised	fwd	discussions	ussr
3	4	donates	patriothole	bc	florida man	obligatory	usa
4	5	jumping	quiz	bce	hmb	available here	leaflet
5	6	odometer	news	happy to	ysk	had to	date unknown
6	7	this	selftitled	decolorized	montage	photoshop	soviet
7	8	lawmaker	announced that	rfakehistoryporn	lpt	swap	wwii
8	9	way the	satire	pareidolia	discussion	michael bay	jap
9	10	shaped	ftw	little guy	florida woman	imgur	pamphlet

Interestingly but logically, in misleading category containing propagandaposters subreddit, "poster", "posters" and "ussr" are the words that influence the prediction the most.

Semantic Embeddings

After that we moved to the stage of extracting deep semantic meaning. To test whether modern neural network NLP technologies can recognize hidden manipulations better than keyword-based search algorithms, we introduced dense vector representations. We selected two compact but powerful language transformers from the **sentence_transformers** library:

- all-MiniLM-L12-v2 (a universal model)
- paraphrase-MiniLM-L3-v2 (specialized in paraphrasing)

All models

Model Name	Performance Sentence Embeddings (14 Datasets)	Performance Semantic Search (6 Datasets)	Avg. Performance	Speed	Model Size
sentence-transformers/all-mpnet-base-v2	69.57	57.02	63.30	2800	420 MB
sentence-transformers/all-distilroberta-v1	68.73	50.94	59.84	4000	290 MB
sentence-transformers/all-MiniLM-L12-v2	68.70	50.82	59.76	7500	120 MB
sentence-transformers/all-MiniLM-L6-v2	68.06	49.54	58.80	14200	80 MB
sentence-transformers/multi-qa-mpnet-base-dot-v1	66.76	57.60	62.18	2800	420 MB
sentence-transformers/multi-qa-distilbert-cos-v1	65.98	52.83	59.41	4000	250 MB
sentence-transformers/paraphrase-multilingual-mpnet-base-v2	65.83	41.68	53.75	2500	970 MB
sentence-transformers/paraphrase-albert-small-v2	64.46	40.04	52.25	5000	43 MB
sentence-transformers/multi-qa-MiniLM-L6-cos-v1	64.33	51.83	58.08	14200	80 MB
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	64.25	39.19	51.72	7500	420 MB
sentence-transformers/paraphrase-MiniLM-L3-v2	62.29	39.19	50.74	19000	61 MB
sentence-transformers/distiluse-base-multilingual-cased-v1	61.30	29.87	45.59	4000	480 MB
sentence-transformers/distiluse-base-multilingual-cased-v2	60.18	27.35	43.77	4000	480 MB

The choice was based on simple speed/performance trade-off. To isolate the influence of the embedder from the classification method, each was tested with default parameters across three algorithms: Logistic Regression (LR), Random Forest (RF), and LightGBM (LGBM). The overall training pipeline looked as follows:

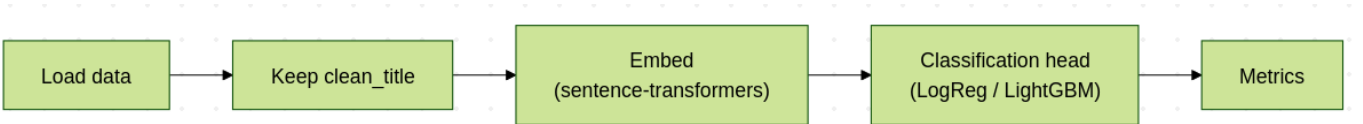


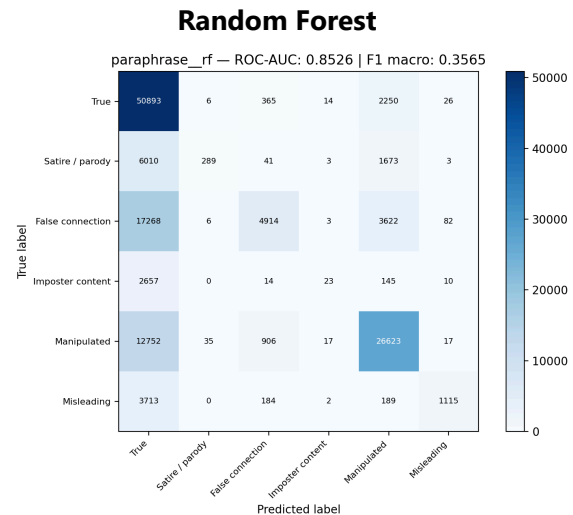
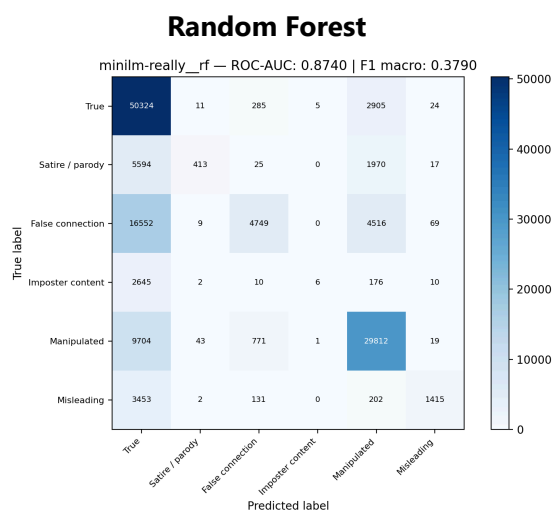
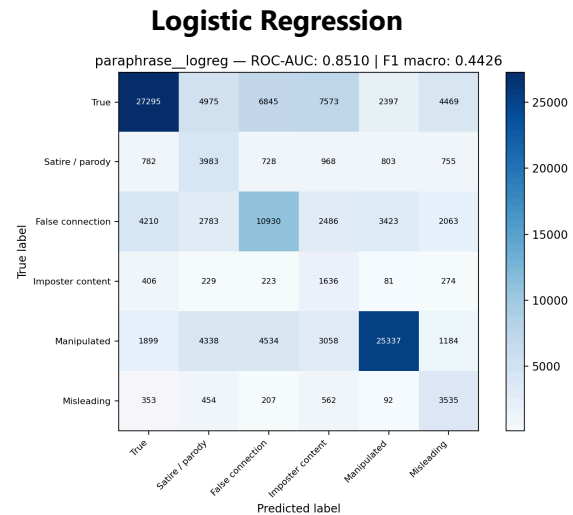
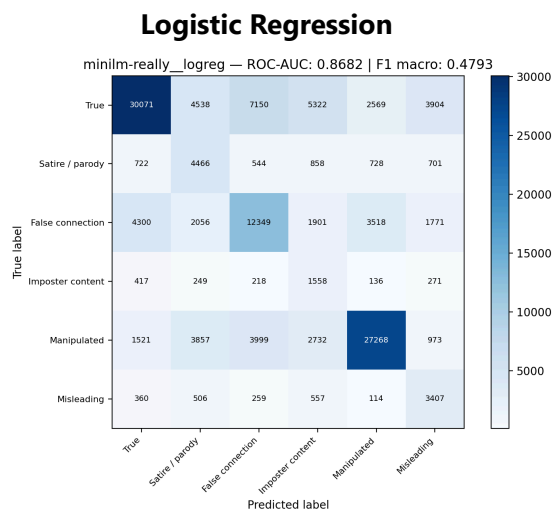
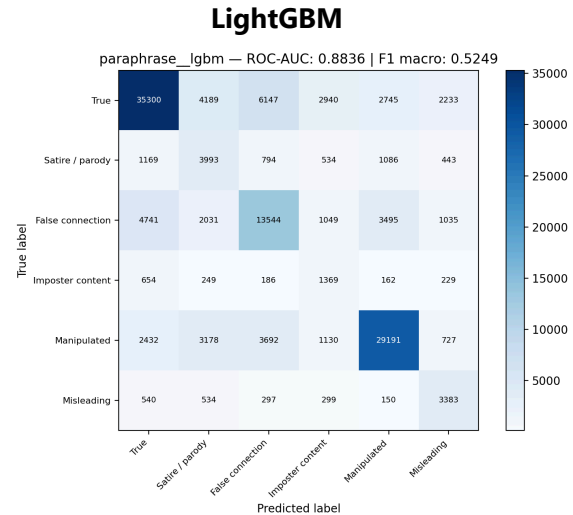
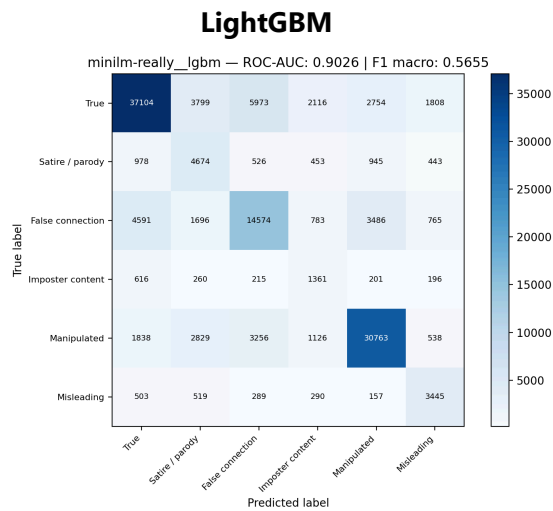
Table: Model performance comparison

Model	ROC AUC	F1 macro	Accuracy	Precision macro	Recall macro
minilm-really_lgbm	0.9026	0.5655	0.6765	0.5405	0.6235
paraphrase_lgbm	0.8836	0.5249	0.6387	0.5030	0.5890
minilm-really_logreg	0.8682	0.4793	0.5823	0.4694	0.5788
paraphrase_logreg	0.8510	0.4426	0.5352	0.4436	0.5517
minilm-really_rf	0.8740	0.3790	0.6383	0.7316	0.3646
paraphrase_rf	0.8526	0.3565	0.6172	0.7005	0.3430

The absolute leader of this stage was the minilm-really_lgbm combination, which achieved an F1-macro of 0.5655, successfully beating our 6-way TF-IDF baseline.

all-MiniLM-L12-v2

paraphrase-MiniLM-L3-v2



Linear algorithms on dense vectors performed noticeably weaker than boosting, failing to separate non-linear semantic boundaries. The most paradoxical results came from the Random Forest algorithm: despite a good overall accuracy, its F1-macro catastrophically collapsed (0.35 - 0.37). A glance at the RF confusion matrices explains why. The algorithm completely capitulated to the dataset imbalance, aggressively predicting majority classes while outputting near zeros for satire and imposter content. On the other hand, gradient boosting (LGBM) proved its architectural superiority. Furthermore, the universal minilm model consistently

outperformed the paraphrase-specific one, likely suggesting that general semantic knowledge is more adaptable to Reddit's structure of the dataset.

Hyperparameter Tuning with Optuna

Afterwards, we've conducted 4 additional training runs with Optuna, all 20 trial long. For the objective function, we strictly utilized the F1-macro metric, deliberately avoiding using Accuracy or F1-weighted because, in the context of our dataset with extremely rare classes (e.g., Imposter Content), those metrics can easily mask poor performance on minority categories. F1-macro, on the contrary, equally accounts for successes in recognizing the dominant "True" category and failures in rare classes, making it the most demanding and honest evaluation criterion.

Table: Optuna. Confusion Matrices

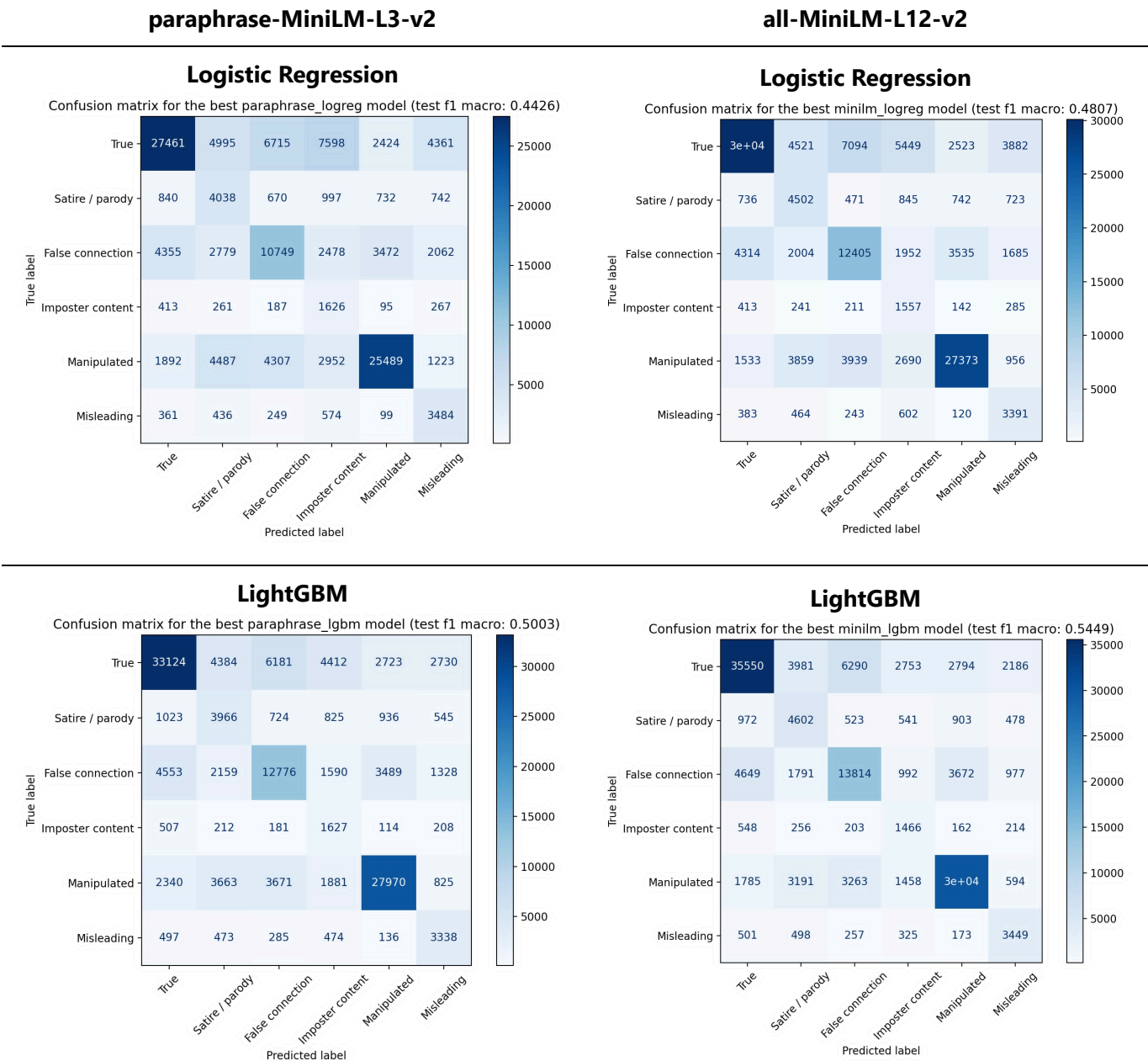
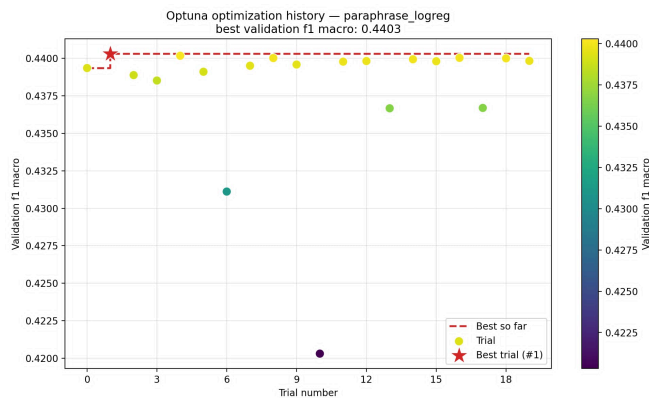


Table: Optuna. Training History

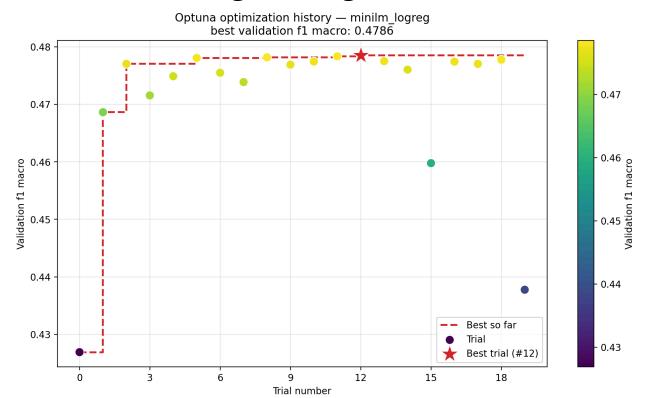
paraphrase-MiniLM-L3-v2

all-MiniLM-L12-v2

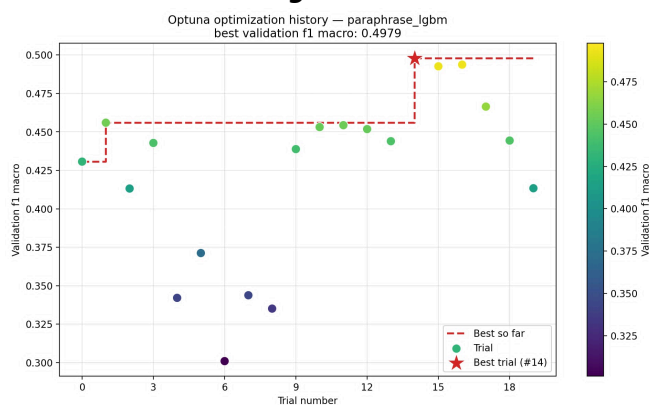
Logistic Regression



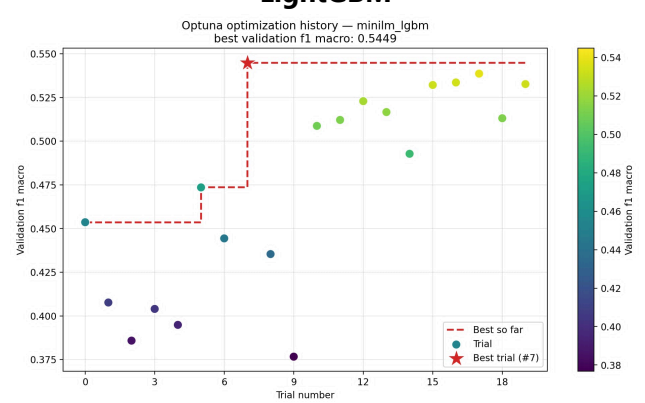
Logistic Regression



LightGBM



LightGBM



The optimization history plots demonstrate that Optuna successfully narrowed the search spaces, discarding ineffective hyperparameter combinations and reaching mathematical possible optimums. Interestingly, after strict hyperparameter tuning, the optimized TF-IDF Baseline slightly overtook the tuned minilm_lgbm in F1-macro (0.5585 vs 0.5449). When comparing the confusion matrices of all tuned models, a consistent pattern emerges: absolutely all classifiers struggle to separate classes that have semantic overlap with truthful news. We observe a significant concentration of false negatives in the True category, indicating that models choose the path of least resistance under conditions of uncertainty.

Table: Tuned models performance comparison

name	accuracy	macro precision	macro recall	macro f1
baseline	0.658	0.534	0.641	0.558
paraphrase_logreg	0.536	0.444	0.551	0.443
paraphrase_lgbm	0.609	0.585	0.585	0.500
minilm_logreg	0.582	0.469	0.579	0.479
minilm_lgbm	0.655	0.520	0.616	0.545

The obtained results suggest that text vectorization methods have reached a plateau of effectiveness. Even after careful hyperparameter tuning and utilizing advanced transformers, the models exhibit consistent error patterns caused by the impossibility of completely separating satire or false connection from truthful

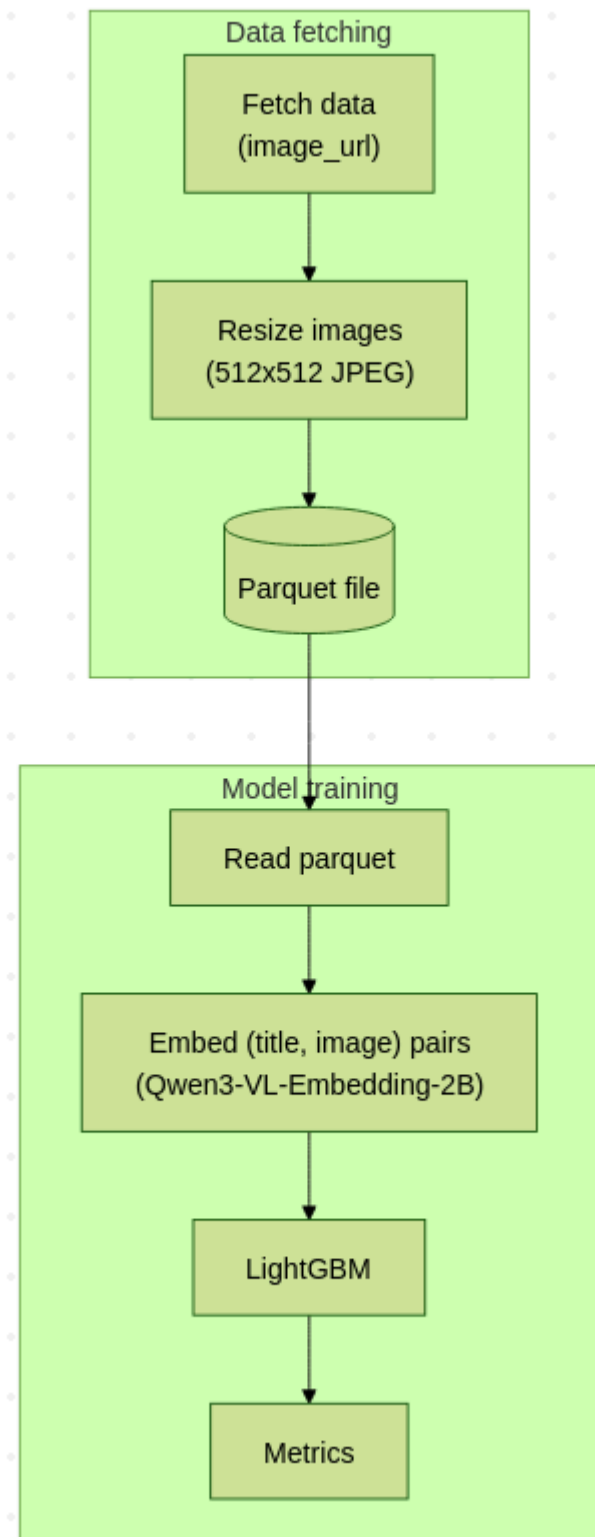
headlines using purely linguistic approach. This confirms the absolute necessity of moving to the final multimodal stage of the project, where image data must be introduced to break this "text-only" barrier.

Tuned models ranked by F1-macro

Model	F1-macro
Baseline (TF-IDF + LR)	0.5585
minilm_lgbm	0.5449
paraphrase_lgbm	0.5003
minilm_logreg	0.4807
paraphrase_logreg	0.4426

Step III. The VLM (Vision-Language Model)

The primary goal of this stage is to overcome the fundamental limitation of text-only classifiers. The technical pipeline for this approach consisted of two stages: data fetching and model training.



The original dataset authors provided over 100gb of image data. Managing this volume proved to be impractical in the cloud computing environments we depended on (such as Kaggle Google Colab mostly). So we opted to fetch the images ourselves using the `image_url` column provided for each sample. During this process, we encountered two significant hindrances. First of all, many images were no longer available on their hosting servers. Secondly, strict rate limiting heavily damaged the download speeds.

Nonetheless, we accepted these limitations to construct a functional multimodal subset. To drastically save on storage space, we have managed to write the data to a parquet file using the following schema:

```
{
    id: str,
    image: bytes,
    clean_title: str,
    split: str,
    6_way_label: int,
}
```

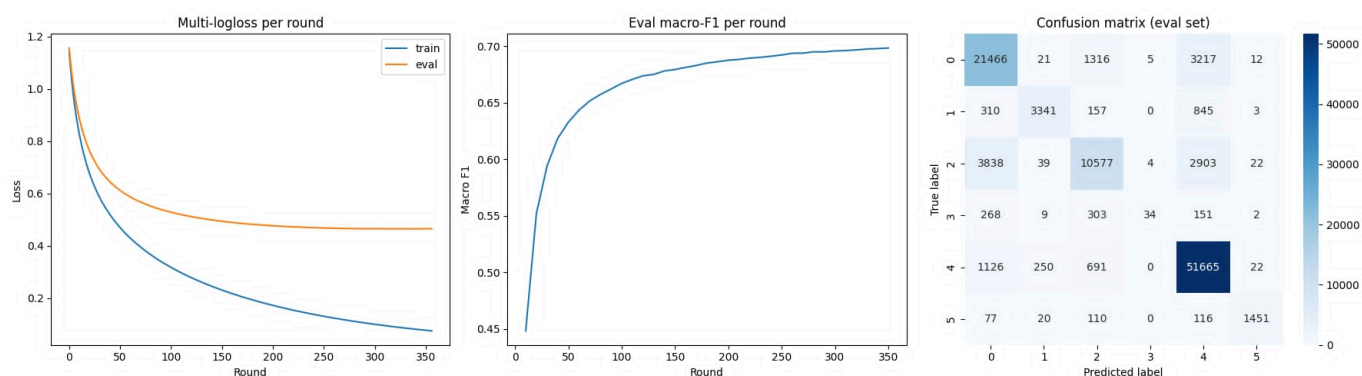
The images were resized to 512x512 pixels and stored directly as binary representations of JPEG files. This approach saved an enormous amount of space. Basically, a file containing over 260k images occupied only about 9.7 GB of storage. Storing the entirety of the image data in this optimized format could be a huge improvement for the project and even for the whole dataset in general.

Conceptually, our multimodal approach is based on extracting dense vector representations (embeddings) from the core of a pretrained VLM, where both the image and the text are projected into a common mathematical space. To extract features from both modalities, the technical implementation utilizes the Qwen architectures (specifically Qwen3-VL-Embedding-2B). The AutoProcessor in this case handles the normalization and preprocessing of the input data. Images are split into patch grids and text is tokenized into a continuous stream.

The following snippet demonstrates the batch embedding extraction process:

```
def embed_batch(clean_titles: list[str], image_bytes_list: list[bytes]) ->
    np.ndarray:
    """Returns float32 array of shape [batch_size, 2048]."""
    inputs = [
        {"text": title or "", "image": Image.open(BytesIO(img)).convert("RGB")}
        for title, img in zip(clean_titles, image_bytes_list)
    ]
    return model.encode(inputs, convert_to_numpy=True, batch_size=len(inputs))
```

The code performs a forward pass through the vision-language encoder, extracts the last hidden layer (hidden_states), applies mean pooling alongside an attention mask to filter out padding tokens and forms a final comprehensive vector in the end. These highly enriched embeddings are then fed into a LightGBM classification head, as gradient boosting previously demonstrated significant architectural superiority over logistic regression methods.



On the reduced dataset with (image, clean_title) pairs we have achieved:

- accuracy: 0.816
- macro F1: 0.764
- macro precision: 0.815
- macro recall: 0.739

Here we have got slightly lower overall accuracy, the macro F1 score noticeably increased to 0.7637. This metric could reflect a significant improvement in separating the more complex categories, such as Satire/Parody (F1: 0.9161) and Misleading (F1: 0.9375), making this model far more balanced and viable for real-world deployment. Basically, much better result than for any of the previous approaches has been achieved. Previously, the best text-only semantic embedders hovered around a macro F1 range of 0.50–0.56. The current multimodal experiment elevated this indicator to 0.70–0.76. This statistically significant increase confirms our main hypothesis: **incorporating the visual modality significantly expands the classifier's capabilities**. The confusion matrices for the multimodal models have become more "diagonal" in some sense, verifying the concentration of correct predictions along the main axis. The consistently high ROC-AUC values (above 0.94) further confirm the model's excellent capacity to rank important semantic objects mathematically in some way. The algorithm now relies confidently on concrete visual evidence rather than purely on statistical lexical patterns or internet slang.

Visual data, in fact, did help indeed.

Implications and Future Directions

Thus to sum up, we have tested machine learning methods of increasing complexity to tackle the task of multimodal misinformation classification. We stumbled upon problems related to the data collection process itself, but we have successfully identified them and considered how they relate to our findings. Additionally, we had our own share of technical challenges with regard to fetching a large number of images from the internet, which we addressed by using recommended techniques for handling large volumes of data.

One of our main goals was to conclusively demonstrate that integrating visual features via a VLM is not merely an architectural complication, it is an absolute necessity for transitioning from word-level analysis to true semantic comprehension of deceptive posts. The implementation of this project is obviously not as powerful, high-quality, or as credible as it could be having more time and capabilities, but(!) it still offers introductory explanatory value for any potential stakeholders. Moreover, it can serve as a solid foundation and valuable contribution. In addition to that, we propose some extra steps in a form of improvement in a case of further modeling/analysis of Fakeddit Data:

1. To extend parquet dataset to contain all the images
2. To try different classification heads.
3. To use metadata such as scores, authors' names etc.